CRYPTOGRAPHY

& INFORMATION SYSTEM SECURITY

Modul praktikum:
Kriptografi Modern (Kunci Asimetris) dan PKI

Tim Asisten Praktikum

2026

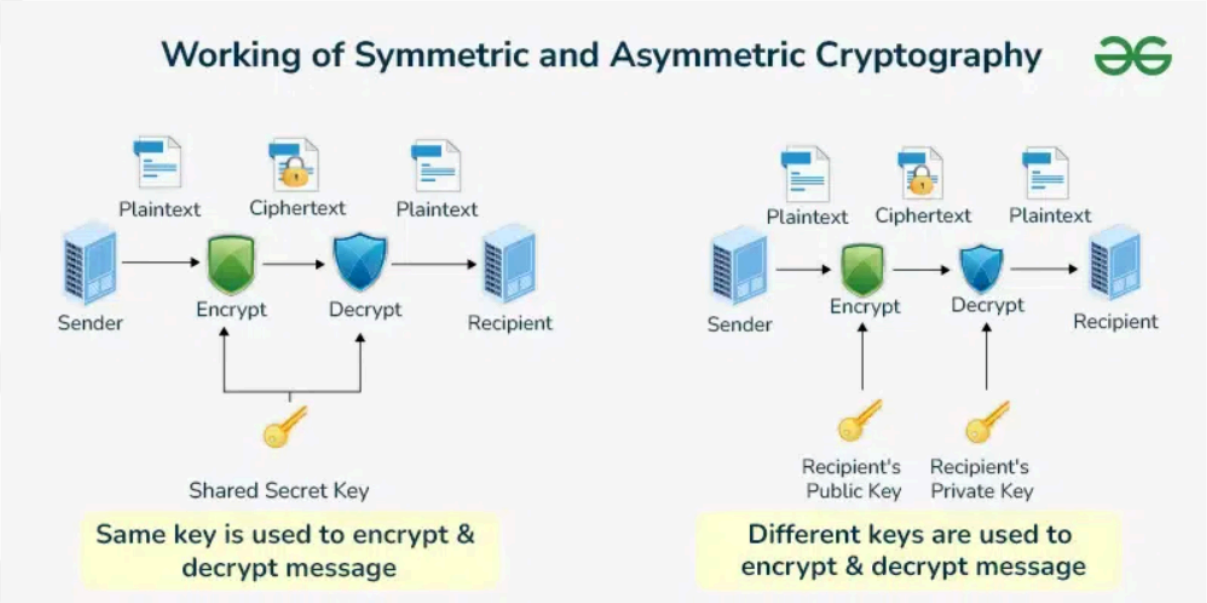
Daftar Isi

Daftar Isi 2
Predecessor 3
Pembangkitan Kunci Asimetris 3
Enkripsi dengan Kunci Simetris 4
RSA 4
Overview 4
Implementasi 5
Diffie-Hellman Key Exchange 5
Overview 5
Langkah-langkah Diffie-Hellman: 5
Implementasi 6
Hubungan dengan TLS 6
PKI 7
Contoh Penerapan: TLS 7
Persiapan Environment 7
Opsi A: Pengguna Linux (langsung) 7
Opsi B: Pengguna Windows/macOS (Docker) 7
Penerapan HTTP tanpa TLS 7
Konfigurasi HTTP dengan TLS 8
Catatan tentang TLS 9
Perintah Docker yang Perlu Diingat 9
Pengantar Fungsi Hash: SHA 10
Apa itu Fungsi Hash? 10
Sifat-sifat fungsi hash: 10
SHA-1 10
Penggunaan dalam Praktik 10
Tugas 11
Daftar Pustaka 12
Tim Asisten Praktikum 12

Predecessor

Pembangkitan Kunci Asimetris

Kunci asimetris adalah dua kunci yang saling berelasi, kunci publik dan kunci privat, yang digunakan untuk melakukan operasi komplemen (Stallings, 2023). Sederhananya, untuk mengenkripsi dan mendekripsi harus menggunakan kunci yang berbeda.



Gambar 1. *Diagram kunci simetris & asimetris*
Sumber: GeeksforGeeks, 2025

Algoritma 1 menunjukkan contoh pendekatan untuk membangkitkan kedua jenis kunci ini.

Algoritma 1. *Contoh Pembangkitan Kunci Asimetris*

```
def cari_fpb(a, b):
    """Mencari FPB (Faktor Persekutuan Terbesar)"""
    while b ≠ 0:
        a, b = b, a % b
    return a

def cari_mod_inverse(e, phi):
    """Mencari Modular Multiplicative Inverse untuk Private Key (d)"""
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d
    return 1

def bangkitkan_kunci(p, q):
    # 1. Menghitung Modulus (n)
    n = p * q

    # 2. Menghitung Totient / Euler Phi
    phi = (p - 1) * (q - 1)

    # 3. Mencari Eksponen Publik (e)
    e = 2
    while e < phi:
        if cari_fpb(e, phi) == 1:
            break
        e += 1
```

```
    # 4. Mencari Eksponen Privat (d)
    d = cari_mod_inverse(e, phi)

    print("=" * 40)
    print("      PROSES PEMBANGKITAN KUNCI        ")
    print("=" * 40)
    print(f"Bilangan Prima p    : {p}")
    print(f"Bilangan Prima q    : {q}")
    print(f"Modulus (n = p*q)   : {n}")
    print(f"Euler Phi (p-1)(q-1): {phi}")
    print("-" * 40)
    print(f"PUBLIC KEY (e, n)   : ({e}, {n})")
    print(f"PRIVATE KEY (d, n)  : ({d}, {n})")
    print("=" * 40)

    return {"n": n, "e": e, "d": d}

if __name__ == "__main__":
    # Gunakan bilangan prima kecil untuk belajar
    # Jangan gunakan angka yang sama!
    p = 61
    q = 53
    bangkitkan_kunci(p, q)
```

Enkripsi dengan Kunci Simetris

Maka untuk menggunakan kunci yang sebelumnya dibangkitkan, enkripsi dapat dilakukan seperti pada Algoritma 2.

Algoritma 2. *Contoh Pembangkitan Kunci Asimetris*

```
def enkripsi(basis, eksponen, n):
    """Modular exponentiation: (basis ^ eksponen) mod n"""
    hasil = 1
    basis = basis % n
    while eksponen > 0:
        if eksponen % 2 == 1:
            hasil = (hasil * basis) % n
        basis = (basis * basis) % n
        eksponen = eksponen // 2
    return hasil

if __name__ == "__main__":
    # Bangkitkan kunci
    kunci = bangkitkan_kunci(61, 53)
    print(f"Kunci E yang didapat: {kunci['e']}")

    pesan = 65

    # Enkripsi dengan kunci publik
    encrypted = enkripsi(pesan, kunci["e"], kunci["n"])
    print(f"Pesan asli    : {pesan}")
    print(f"Hasil Enkripsi: {encrypted}")
```

Challenge!
Cobalah untuk mendekripsikan cipher text!

RSA

Overview

Skema RSA adalah salah satu skema yang plain text dan cipher text dipandang sebagai bilangan bulat antara 0 dan n-1 untuk suatu nilai n (Stallings, 2023). Sebetulnya, algoritma RSA sudah dijalankan pada *Predecessor*. Jika diformulakan, algoritma tersebut tampak seperti pada Gambar 2.

Key Generation by Alice	
Select p, q	p and q both prime, $p \neq q$
Calculate $n = p \times q$	
Calculate $\phi(n)=(p-1)(q-1)$	
Select integer e	$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$
Calculate d	$d \equiv e^{-1} \pmod{\phi(n)}$
Public key	$PU = \{e, n\}$
Private key	$PR = \{d, n\}$

Encryption by Bob with Alice's Public Key	
Plaintext:	$M < n$
Ciphertext:	$C = M^e \bmod n$

Decryption by Alice with Alice's Private Key	
Ciphertext:	C
Plaintext:	$M = C^d \bmod n$

Gambar 2. *Algortima RSA*
Sumber: Stallings, 2023

Implementasi

Implementasi RSA dalam Python sudah ditunjukkan pada Algoritma 1 dan Algoritma 2 di bagian Predecessor. Gabungkan kedua algoritma tersebut dalam satu file rsa.py untuk mendapatkan program RSA yang lengkap: pembangkitan kunci, enkripsi, dan dekripsi.

Challenge!
Cobalah untuk mengenkripsi string!

Diffie-Hellman Key Exchange

Overview

Diffie-Hellman (DH) adalah protokol pertukaran kunci yang memungkinkan dua pihak menyepakati sebuah kunci rahasia bersama (shared secret) melalui saluran komunikasi yang tidak aman, tanpa perlu mengirimkan kunci rahasia itu sendiri (Stallings, 2023).

Prinsip dasarnya memanfaatkan sifat matematika dari modular exponentiation: mudah dihitung ke satu arah, tetapi sangat sulit untuk dibalikkan (discrete logarithm problem).

Langkah-langkah Diffie-Hellman:

1. Alice dan Bob menyepakati parameter publik: bilangan prima p dan generator g.
2. Alice memilih kunci privat a (rahasia), menghitung kunci publik A = g^a mod p, dan mengirim A ke Bob.
3. Bob memilih kunci privat b (rahasia), menghitung kunci publik B = g^b mod p, dan mengirim B ke Alice.
4. Alice menghitung shared secret: s = B^a mod p.
5. Bob menghitung shared secret: s = A^b mod p.
6. Kedua nilai s akan selalu sama, karena g^(ab) mod p = g^(ba) mod p.

Penyadap yang mengamati saluran komunikasi hanya melihat p, g, A, dan B. Untuk mendapatkan shared secret, penyadap harus menyelesaikan masalah discrete logarithm, yang secara komputasi sangat sulit untuk bilangan prima yang besar.

Implementasi

Buat file **diffie_hellman.py** dan masukkan kode berikut:

Algoritma 3. *Diffie-Hellman Key Exchange*

```
def diffie_hellman():
    # Parameter publik (disepakati bersama)
    p = 23   # bilangan prima
    g = 5    # generator

    # Kunci privat masing-masing (rahasia)
    a = 6    # kunci privat Alice
    b = 15   # kunci privat Bob

    # Kunci publik yang dikirimkan
    A = pow(g, a, p)   # Alice kirim ke Bob
    B = pow(g, b, p)   # Bob kirim ke Alice

    # Shared secret (dihitung masing-masing)
    secret_alice = pow(B, a, p)   # Alice hitung
    secret_bob   = pow(A, b, p)   # Bob hitung

    print("=" * 40)
    print("       DIFFIE-HELLMAN KEY EXCHANGE      ")
    print("=" * 40)
    print(f"Parameter publik: p={p}, g={g}")
    print(f"Kunci privat Alice (a): {a}")
    print(f"Kunci privat Bob   (b): {b}")
    print("-" * 40)
    print(f"Kunci publik Alice (A = g^a mod p): {A}")
    print(f"Kunci publik Bob   (B = g^b mod p): {B}")
    print("-" * 40)
    print(f"Shared secret Alice (B^a mod p): {secret_alice}")
```

```
    print(f"Shared secret Bob   (A^b mod p): {secret_bob}")
    print(f"Kunci sama? {secret_alice == secret_bob}")
    print("=" * 40)

if __name__ == "__main__":
    diffie_hellman()
```

Hubungan dengan TLS

Diffie-Hellman merupakan salah satu mekanisme pertukaran kunci yang digunakan dalam protokol TLS. Pada TLS 1.3, pertukaran kunci wajib menggunakan Ephemeral Diffie-Hellman (DHE) atau Elliptic Curve Diffie-Hellman Ephemeral (ECDHE), yang membangkitkan pasangan kunci baru untuk setiap sesi. Hal ini menjamin forward secrecy: meskipun kunci privat server bocor di kemudian hari, sesi-sesi lama yang sudah tercatat tetap tidak dapat didekripsi (Rescorla, 2018).

PKI

PKI dalam hal ini tidak merujuk pada Partai Komunis Indonesia, tetapi **Public Key Infrastructure**. PKI adalah sekumpulan aturan dan kebijakan yang dibutuhkan untuk membuat, mendistribusikan, menggunakan, menyimpan, mengelola, dan membuang sertifikat digital. **Sertifikat digital** adalah dokumen elektronis yang digunakan untuk membuktikan **kepemilikan kunci publik**.

Contoh Penerapan: TLS

Contoh penerapan PKI adalah penerapan protokol Transport Layer Security (Rescorla, 2018) pada protokol HTTP (Thomson & Benfield, 2022). Untuk keperluan praktikum, gunakan **linux pribadi** ataupun **Docker**. Instruksi untuk kedua opsi disediakan di bawah.

Persiapan Environment

Opsi A: Pengguna Linux (langsung)

Buka terminal dan install paket yang dibutuhkan:

```
sudo apt update
sudo apt install -y nginx openssl tcpdump curl
```

Opsi B: Pengguna Windows/macOS (Docker)

Pastikan Docker Desktop sudah terinstal dan berjalan. Buat folder praktikum, lalu buat file Dockerfile (tanpa extension):

```
FROM ubuntu:24.04

RUN apt-get update && apt-get install -y \
    nginx openssl tcpdump curl nano \
    && rm -rf /var/lib/apt/lists/*

EXPOSE 80 443
CMD ["bash"]
```

Build dan jalankan container:

```
docker build -t praktikum-tls .
docker run -it --name tls-lab --cap-add=NET_ADMIN praktikum-tls
```

Penerapan HTTP tanpa TLS

1. Buat project sederhana:

```
mkdir -p /var/www/html/project_name
echo "<h1>Halo Praktikum TLS</h1>" > /var/www/html/project_name/index.html
```

2. Buat konfigurasi nginx. Buka file konfigurasi:

```
nano /etc/nginx/sites-enabled/project_name
```

Masukkan konfigurasi berikut:

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /var/www/html/project_name;
        index index.html;
    }
}
```

3. Jalankan nginx dan uji HTTP:

```
service nginx start
curl -v http://localhost
```

4. Capture paket dengan tcpdump. Buka terminal kedua (untuk Docker: `docker exec -it tls-lab bash`):

```
tcpdump -i any -A port 80
```

Lalu di terminal pertama, jalankan curl lagi. Perhatikan bahwa data dikirim dalam bentuk plain text yang bisa dibaca.

5. Coba akses dengan HTTPS:

```
curl -v https://localhost
```

Perhatikan bahwa koneksi akan gagal karena belum ada konfigurasi TLS.

Konfigurasi HTTP dengan TLS

1. Buat folder untuk sertifikat dan generate private key:

```
mkdir -p /etc/nginx/ssl
openssl genrsa -out /etc/nginx/ssl/server.key 2048
```

2. Buat Certificate Signing Request (CSR):

```
openssl req -new -key /etc/nginx/ssl/server.key -out /etc/nginx/ssl/server.csr
```

Isi data yang diminta (boleh tekan Enter untuk menggunakan nilai default).

3. Generate sertifikat self-signed:

```
openssl x509 -req -days 365 \
  -in /etc/nginx/ssl/server.csr \
  -signkey /etc/nginx/ssl/server.key \
  -out /etc/nginx/ssl/server.crt
```

4. Edit konfigurasi nginx untuk menambahkan HTTPS:

```
nano /etc/nginx/sites-enabled/project_name
```

Ganti seluruh isi file dengan konfigurasi berikut:

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /var/www/html/project_name;
        index index.html;
    }
}

server {
    listen 443 ssl;
    server_name localhost;

    ssl_certificate     /etc/nginx/ssl/server.crt;
    ssl_certificate_key /etc/nginx/ssl/server.key;

    location / {
        root /var/www/html/project_name;
        index index.html;
    }
}
```

5. Restart nginx dan uji TLS:

```
service nginx restart
curl -vk https://localhost
```

Flag **-k** pada curl digunakan untuk mengizinkan sertifikat self-signed. Perhatikan output curl yang menunjukkan informasi TLS handshake.

6. Capture paket HTTPS dengan tcpdump:

```
tcpdump -i any -A port 443
```

Bandingkan dengan capture HTTP sebelumnya. Perhatikan bahwa data HTTPS terenkripsi dan tidak bisa dibaca.

Catatan tentang TLS

TLS versi terbaru saat ini adalah 1.3 dengan autentikasinya bisa menggunakan kriptografi asimetris ataupun simetris PSK (Rescorla, 2018). Untuk kriptografi asimetrisnya adalah RSA, ECDSA, atau EdDSA. Salah satu perbedaan TLS versi 1.3 dan 1.2 adalah TLS 1.3 tidak lagi menggunakan static RSA dan static Diffie-Hellman, melainkan mewajibkan ephemeral key exchange (DHE/ECDHE) untuk menjamin forward secrecy.

Perintah Docker yang Perlu Diingat

Bagi pengguna Docker, berikut perintah-perintah yang berguna selama praktikum:

```
# Keluar dari container tanpa menghapus
exit

# Masuk kembali ke container
docker start -i tls-lab

# Buka terminal kedua ke container
docker exec -it tls-lab bash

# Hapus container (reset total)
docker rm -f tls-lab
```

Pengantar Fungsi Hash: SHA

Apa itu Fungsi Hash?

Fungsi hash kriptografi adalah fungsi matematika yang menerima input dengan panjang sembarang dan menghasilkan output dengan panjang tetap (disebut hash value, digest, atau fingerprint). Fungsi ini bersifat satu arah: mudah menghitung hash dari sebuah input, tetapi secara komputasi tidak layak untuk menemukan input dari sebuah hash (Stallings, 2023).

Sifat-sifat fungsi hash:

- **Pre-image resistance**: diberikan hash h, sulit menemukan pesan m sehingga hash(m) = h.
- **Second pre-image resistance**: diberikan pesan m1, sulit menemukan m2 ≠ m1 sehingga hash(m1) = hash(m2).
- **Collision resistance**: sulit menemukan dua pesan berbeda m1 dan m2 sehingga hash(m1) = hash(m2).

SHA-1

SHA-1 (Secure Hash Algorithm 1) menghasilkan digest sepanjang 160 bit (20 byte). SHA-1 diperkenalkan oleh NIST pada tahun 1995 dan sempat menjadi standar yang digunakan secara luas, termasuk dalam sertifikat TLS dan Git.

Namun, pada tahun 2017 peneliti dari Google dan CWI Amsterdam berhasil mendemonstrasikan serangan collision terhadap SHA-1 (SHAttered). Sejak saat itu, SHA-1 dianggap **tidak lagi aman** untuk penggunaan kriptografi dan telah ditinggalkan dari sebagian besar standar keamanan.

Penggunaan dalam Praktik

Fungsi hash digunakan di banyak tempat dalam keamanan informasi: verifikasi integritas data, digital signature, penyimpanan password, HMAC (Hash-based Message Authentication Code), dan sebagai komponen dalam protokol TLS yang sudah dipraktikkan sebelumnya.

Implementasi dan eksplorasi lebih lanjut mengenai fungsi hash akan dilakukan pada modul praktikum berikutnya.

Tugas

1. Jalankan Algoritma 1 dan 2 (implementasi RSA) dengan mengganti nilai **p** dan **q** dengan bilangan prima pilihan Anda sendiri (keduanya harus berbeda dari contoh dan berbeda satu sama lain). Gunakan pesan berupa bilangan bulat yang kurang dari **n**. Lakukan proses enkripsi dan dekripsi, lalu pastikan hasil dekripsi sama dengan pesan awal. Screenshot output program Anda
   Output format: source code Python (.py) dan screenshot output dalam .pdf
2. Jalankan Algoritma 3 (implementasi Diffie-Hellman) dengan mengganti nilai **p**, **g**, **a**, dan **b** dengan nilai pilihan Anda sendiri (pastikan **p** adalah bilangan prima dan **g** kurang dari **p**). Verifikasi bahwa shared secret yang dihasilkan Alice dan Bob bernilai sama. Jelaskan secara singkat: mengapa penyadap yang mengetahui p, g, A, dan B tidak dapat menghitung shared secret?
   Output format: source code Python (.py), screenshot output, dan penjelasan singkat dalam .pdf
3. Lakukan praktikum TLS sesuai instruksi pada bagian PKI (menggunakan Linux atau Docker). Dokumentasikan langkah-langkah berikut dengan screenshot:
   - Hasil curl -v http://localhost (HTTP tanpa TLS).
   - Hasil capture tcpdump pada port 80 (tunjukkan bahwa data terlihat dalam plain text).
   - Hasil curl -vk https://localhost (HTTP dengan TLS).
   - Hasil capture tcpdump pada port 443 (tunjukkan bahwa data terenkripsi).

   Berikan penjelasan singkat mengenai perbedaan yang Anda amati antara HTTP dan HTTPS dari sisi keamanan data yang dikirimkan.
   Output format: screenshot langkah-langkah dan penjelasan dalam .pdf
4. Berdasarkan materi pengantar SHA-1 pada modul ini dan sumber-sumber lainnya, jelaskan dengan bahasa Anda sendiri: apa itu fungsi hash kriptografi, mengapa SHA-1 tidak lagi dianggap aman, dan sebutkan minimal dua contoh penggunaan fungsi hash dalam kehidupan nyata (misalnya dalam sistem atau aplikasi yang Anda gunakan sehari-hari).
   Output format: penjelasan dalam .pdf

Deadline:
C1: 19/04/2026 23:59 WIB
C2: 20/04/2026 23:59 WIB

Link Pengumpulan

Daftar Pustaka

Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3* (No. RFC8446; hlm. RFC8446). RFC Editor. https://doi.org/10.17487/RFC8446

Stallings, W. (2023). *Cryptography and network security: Principles and practice* (Eighth edition, global edition). Pearson.

Thomson, M., & Benfield, C. (Ed.). (2022). *HTTP/2* (No. RFC9113; hlm. RFC9113). RFC Editor. https://doi.org/10.17487/RFC9113

GeeksforGeeks. (2025). Difference Between Symmetric and Asymmetric Key Encryption. GeeksforGeeks. https://www.geeksforgeeks.org/computer-networks/difference-between-symmetric-and-asymmetric-key-encryption/

Tim Asisten Praktikum

1. Franklin
2. Lyan
3. **Ichsan (PIC Praktikum)**
4. Alvin
5. **Radhi (PIC Praktikum)**